Univerzita Palackého
v Olomouci

Praktikum z programování 2
Protokol č. 2

Téma: Výběr tématu, cílové skupiny a popis plánovaného řešení a zdrojů

Vyučující: doc. RNDr. Petr Šaloun, Ph.D., Katedra technické a informační výchovy, PdF UPOL

Jméno a příjmení studenta: Vojtěch Němčák

Datum vypracování: 4.4.2026

Témata k prostudování: Při vývoji aplikace byla prostudována následující témata a dokumentace:

- Python – práce s GUI knihovnou tkinter (widgety, události, TopLevel okna)
- pandas – načítání a zpracování dat z Excelu (pd.read_excel, DataFrame operace)
- matplotlib – tvorba grafů, integrace s tkinter (FigureCanvasTkAgg, NavigationToolbar2Tk)
- openpyxl – čtení struktury xlsx souborů (ExcelFile, sheet_names)
- Regulární výrazy (modul re) – parsování rozsahových pořadí (napr. 11-15)
- json – ukládání uživatelského nastavení (téma, historie hledání)
- Dokumentace CSU – formáty souborů s nejčetnějšími jmény

Moodle k předmětu PP2, výukové materiály k Pythonu, zejména linky a knihovny

Zadání:

Na základě popisu zadání schváleného a případně doplněného a komentovaného vyučujícím a sepsaného v Protokolu 1 popište řešení a zdokumentujte program, testovací vstupní a výstupní data (nezapomeňte je přibalit do dodevzdávaného ZIPu) a popište/diskutujte použité algoritmy a postupy tak, aby byl postup jasný a doložili jste funkčnost programu i použité knihovny, algoritmy a postupy a tyto případně své řešení porovnali s jiným dostupným řešením.
Součástí řešení může být například .txt soubor s linky na zdroje, které vás inspirovaly, popis github prostředí, odkud si stáhnete zdrojové kódy, licence… na svůj počítač a použijete je pro řešení… spolu se zdrojovým textem(-y) programu, testovacími daty s tímto protokolem 2 pojmenujte podle pokynů popsaných níže) zabalte do jediného ZIP archivu pojmenovaného *ID-2.zip*, jde ID je Vaše školní ID, např. *novakp-2.zip*. Nezapomeňte například do .txt souboru přidat soubor nutných knihoven v zápisu pro „pip“, usnadňující/definující instalační skript.
Odevzdání druhého protokolu ID-2.ZIP bude pravděpodobně obsahovat soubory:

- ID-protokol-PP2-2.odt (případně .docx) je tento soubor pojmenovaný podle vzoru, a vyplněný podle požadavků, doplněný, screenshoty obrazovek a vaším komentářem, linky na zdroje, dokumentaci, popisy a návody, které jste jste našli a používáte jako východisko/inspiraci, použili a nebo použít plánujete.

Kontakt: petr.saloun@upol.cz

Výsledek konzultujte v případě potřeby ve cvičení s vyučujícím. Výsledek odevzdávejte výhradně do moodle pro daný předmět a protokol 2. Jiný způsob odevzdání nebude akceptován.

Návrh řešení:

Architektura

Aplikace je implementována jako dvě třídy: AutocompleteDialog (dialog s našeptávačem) a JmenaApp (hlavní logika a GUI). Data jsou uchovávána v pandas DataFrame v paměti. Persistentní uložení (téma, historie) využívá JSON soubory v domovském adresáři uživatele.

Detekce formátu souboru

CSU publikuje data ve dvou odlišných formátech:

- Formát A – oddělené listy Chlapci / Dívky, sloupce: Pořadí v CR + Jméno
- Formát B – jeden list s kluky a holkami vedle sebe (Pořadí + Kluci jména + Pořadí + Holky jména)

Formát je detekován automaticky metodou detect_format() na základě názvů listů nebo počtu sloupců s pořadím/jmény v záhlaví.

Výsledky a fakta

Použité knihovny

pandas	Načítání xlsx, manipulace s DataFrame, groupby, drop_duplicates
matplotlib	Vykreslování grafů, integrace do tkinter přes FigureCanvasTkAgg
tkinter	Grafické uživatelské rozhraní (vestavěná knihovna Pythonu)
openpyxl	Čtení xlsx souborů (závislost pandas)
re	Regulární výrazy – parsování pořadí, čištění jmen
json	Persistence nastavení a historie hledání
math	Exponenciální výpočet pravděpodobnosti setkání

Testovací data a výstupy

- 100_nejcetnejsich_detskych_jmen_republika_2022.xlsx – Formát A, 100 jmen každého pohlaví
- 100_nejcetnejsich_detskych_jmen_republika_2024.xlsx – Formát A, obsahuje NEZJISTENO – filtrován
- tabulka_s_celkovym_poradim_2018.xlsx – Formát B, rozsahy pořadí až 247–680
- Top 10 kluků 2024: Jakub, Matyáš, Jan, Adam, Matěj, David, Vojtěch, Filip, Dominik, Tomáš
- Top 10 holek 2024: Eliška, Viktorie, Sofie, Anna, Natálie, Amálie, Ema, Tereza, Laura, Adéla

Vysvětlení kódu po částech

1. Importy a globální konstanty

Na začátku souboru jsou importovány všechny potřebné knihovny a definovány globální konstanty pro cesty k JSON souborům a slovník barevných témat.

```
import pandas as pd
import matplotlib.pyplot as plt
import tkinter as tk
from tkinter import filedialog, messagebox, simpledialog
import os, re, math, json
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk


HISTORIE_SOUBOR = os.path.join(os.path.expanduser('~'), '.jmena_historie.json')
TEMA_SOUBOR     = os.path.join(os.path.expanduser('~'), '.jmena_tema.json')
```

os.path.expanduser('~') vrátí cestu k domovskému adresáři uživatele (např. C:/Users/Jmeno na Windows). Soubory s historií a tématem se tak ukládají trvale mezi spuštěními aplikace.

Slovník TEMA definuje barvy pro světlý a tmavý režim jako Python slovník, ze kterého se dynamicky čtou hodnoty při sestavování UI:

```
TEMA = {
    'svetly': { 'bg': '#f5f5f5', 'fg': '#1a1a1a', 'btn_bg': '#ffffff', ... },
    'tmavy':  { 'bg': '#1e1e2e', 'fg': '#cdd6f4', 'btn_bg': '#313244', ... },
}
```

2. Třída AutocompleteDialog

Tato třída dědí z tk.Toplevel a implementuje modální dialog s našeptávačem. Dědičnost z Toplevel zajistí, že dialog je samostatné okno blokující interakci s hlavním oknem (grab_set()).

```
class AutocompleteDialog(tk.Toplevel):
    def __init__(self, parent, titul, prompt, moznosti, tema, vice=False):
        super().__init__(parent)    # inicializuje tkinter okno
        self.grab_set()             # zablokuje ostatní okna (modální chování)
        self.moznosti = [m.capitalize() for m in moznosti]
```

Metoda _aktualizuj_navrhy

Volá se automaticky při každé změně textu v poli (trace_add). Filtruje seznam jmen podle toho, co uživatel napsal, a zobrazí max 8 návrhů v listboxu. Při zadávání více jmen (vice=True) našeptává jen poslední jméno za posledním oddělovačem čárkou:

```
def _aktualizuj_navrhy(self, *_):
    text = self.entry_var.get()
    if self.vice:                               # pro více jmen oddělených čárkou
        casti = [c.strip() for c in text.split(',')]
        hledat = casti[-1]                      # našeptáváme jen poslední část
    else:
        hledat = text
    navrhy = [m for m in self.moznosti
              if m.upper().startswith(hledat.upper())][:8]  # max 8 návrhů
    self.listbox.delete(0, 'end')               # vymaž starý obsah
    for n in navrhy:
        self.listbox.insert('end', n)
```

3. Třída JmenaApp – inicializace a sestavení UI

Konstruktor __init__ nastaví základní atributy a zavolá _sestav_ui(). UI je záměrně odděleno do vlastní metody, protože ji potřebujeme volat znovu při přepnutí tématu (celé UI se přestaví od základů).

```
class JmenaApp:
    def __init__(self, root):
        self.data = pd.DataFrame()    # sem se uloží všechna načtená data
        self.seznam_roku = []         # seznam dostupných roků
        self.tema_klic = self._nacti_tema()    # 'svetly' nebo 'tmavy'
        self.historie = self._nacti_historii() # seznam posledních hledání
        self._sestav_ui()

    def _sestav_ui(self):
        t = TEMA[self.tema_klic]      # aktivní barevné schéma
        for w in self.root.winfo_children():
            w.destroy()               # smaž staré widgety při přepnutí tématu
        # ... vytvoření tlačítek, labelů atd.
```

4. Persistentní nastavení – téma a historie

Téma a historie hledání se ukládají do JSON souborů v domovském adresáři, takže se pamatují mezi spuštěními. Použití try/except zajistí, že chybějící soubor při prvním spuštění nevyhodí chybu:

```
def _nacti_tema(self):
    try:
        with open(TEMA_SOUBOR) as f:
            return json.load(f).get('tema', 'svetly')  # default = svetly
    except Exception:
        return 'svetly'   # soubor neexistuje – použij výchozí

def _uloz_historii(self):
    with open(HISTORIE_SOUBOR, 'w') as f:
        json.dump(self.historie[-50:], f, ensure_ascii=False)  # max 50 záznamů
```

5. Parsery – automatická detekce formátu souboru

Metoda detect_format() nejprve zkontroluje názvy listů. Pokud žádný neobsahuje klíčové slovo pohlaví, prohledá záhlaví listu a hledá alespoň 2 sloupce s pořadím nebo jmény – to signalizuje Formát B:

```
def detect_format(self, f):
    xls = pd.ExcelFile(f)
    sheets_lower = [s.lower() for s in xls.sheet_names]
    # Formát A: listy se jmenují Chlapci/Dívky/...
    gender_kw = ['chlapci', 'divky', 'kluci', 'holky', 'boys', 'girls']
    if any(kw in sl for sl in sheets_lower for kw in gender_kw):
        return 'oddelene_listy'
    # Formát B: jeden list se dvěma sloupci pořadí vedle sebe
    for sheet in xls.sheet_names:
        df = pd.read_excel(f, sheet_name=sheet, header=None, nrows=5)
        for _, row in df.iterrows():
            vals = [str(v).lower() for v in row]
            if sum(1 for v in vals if 'porad' in v) >= 2:
                return 'vedle_sebe'
    return 'oddelene_listy'  # výchozí
```

Kontakt: petr.saloun@upol.cz

6. Parser formátu A – oddělené listy

Projde všechny listy souboru, u každého určí pohlaví z názvu listu, najde řádek s hlavičkou (hledá slovo 'jméno') a načte data. Klíčová část je dynamické nalezení správných sloupců – nepředpokládáme pevné názvy ani pořadí:

```
def nacti_format_oddelene_listy(self, f, rok):
    for sheet in xls.sheet_names:
        # Detekce pohlaví z názvu listu
        if any(x in sheet.lower() for x in ['chlapci', 'kluc', 'boy']):
            pohlavi = 'Kluk'
        elif any(x in sheet.lower() for x in ['divky', 'holky', 'girl']):
            pohlavi = 'Holka'
        else:
            continue   # neznámý list, přeskočíme

        # Nalezení řádku s hlavičkou (hledáme 'jméno')
        header_row = next(
            (i for i, row in df.iterrows()
             if any('jmeno' in str(v).lower() for v in row)), None
        )

        # Dynamické nalezení sloupců – funguje i při jiném pořadí/názvu
        col_jmeno  = next((c for c in cols if 'jmeno' in c.lower()), None)
        col_poradi = next((c for c in cols if 'porad' in c.lower()), None)
```

7. Parser formátu B – kluci a holky vedle sebe

Klíčový algoritmus: pro každý sloupec s pořadím najde nejbližší sloupec s jmény napravo (podle fyzického indexu v tabulce). Tím správně spáruje Pořadí kluků s Kluci jména a Pořadí holek s Holky jména, bez ohledu na mezilehlé prázdné sloupce:

```
def nacti_format_vedle_sebe(self, f, rok):
    # Indexy sloupců podle obsahu záhlaví
    poradi_idxs = [i for i, c in enumerate(cols) if 'porad' in c.lower()]
    jmena_idxs  = [i for i, c in enumerate(cols) if 'jmen'  in c.lower()]

    pairs = []
    for pi in poradi_idxs:
        # Hledáme nejbližší jméno-sloupec NAPRAVO od tohoto pořadí-sloupce
        candidates = [ji for ji in jmena_idxs if ji > pi]
        ji = min(candidates)   # nejbližší = nejmenší index větší než pi
        cp, cj = cols[pi], cols[ji]

        # Pohlaví z názvu jméno-sloupce
        combined = (cp + cj).lower()
        if any(x in combined for x in ['kluc', 'chlapci']): pohlavi = 'Kluk'
        elif any(x in combined for x in ['holky', 'divky']): pohlavi = 'Holka'
        else: pohlavi = 'Kluk' if len(pairs) == 0 else 'Holka'  # fallback
        pairs.append((cp, cj, pohlavi))
```

8. Pomocné metody – dekoduj_poradi a cisti_jmeno

dekoduj_poradi() zpracuje hodnotu pořadí z buňky. Pokud je to rozsah jako '11-15', vypočítá průměr (13) – to zajistí jeden bod v grafu místo duplicit. Regulární výraz \d+ najde všechna čísla v řetězci:

```
def dekoduj_poradi(self, x):
    s = str(x).replace('–', '-').replace(' ', '')
    match = re.findall(r'\d+', s)   # najde všechna čísla
    if len(match) > 1:
        # Rozsah '11-15' -> průměr round((11+15)/2) = 13
        return [round((int(match[0]) + int(match[-1])) / 2)]
    return [int(match[0])]
```

cisti_jmeno() odstraní z jména vše kromě písmen (včetně diakritiky) a vrátí uppercase. Zároveň filtruje placeholder hodnoty jako NEZJISTENO, které se v datech CSU vyskytují:

```
IGNOROVAT_JMENA = {'NEZJISTENO', 'NEUVEDENO', 'NEZNAME', ...}


def cisti_jmeno(self, jmeno):
    # Regulární výraz: ponech jen česká a anglická písmena a mezery
    vysledek = re.sub(
        r'[^a-zA-ZáčďéěíňóřšťúůýžÁČDEINoRSTUYZ\s]', '', str(jmeno)
    ).strip().upper()
    return '' if vysledek in self.IGNOROVAT_JMENA else vysledek
```

9. Načtení dat – metoda nacti_data

Tato metoda nejprve vymaže stará data (self.data = pd.DataFrame()), pak projde všechny vybrané soubory, pro každý extrahuje rok z názvu souboru regulárním výrazem, detekuje formát a zavolá správný parser. Na konci sloučí všechny DataFrame do jednoho a odstraní duplicity:

```
def nacti_data(self):
    self.data = pd.DataFrame()     # RESET – vymaž stará data
    vsechny_seznamy = []


    for f in files:
        # Extrakce roku z názvu souboru: '...2022.xlsx' -> 2022
        rok_match = re.findall(r'\d{4}', os.path.basename(f))
        rok = int(rok_match[0]) if rok_match else 2024


        fmt = self.detect_format(f)
        rows = (self.nacti_format_oddelene_listy(f, rok)
                if fmt == 'oddelene_listy'
                else self.nacti_format_vedle_sebe(f, rok))
        vsechny_seznamy.append(pd.DataFrame(rows))


    # Sloucení všech DataFrames + odstranění duplicit
    self.data = pd.concat(vsechny_seznamy, ignore_index=True)
                .drop_duplicates(subset=['Pořadí', 'Jméno', 'Pohlaví', 'Rok'])
```

10. Historický graf – metoda _kresli_graf

Metoda vytvoří matplotlib figure a nakreslí jednu čáru pro každou kombinaci jméno+pohlaví. Každá série dostane unikátní barvu, styl čáry a marker z předpřipravených seznamů (rotací přes modulo). Graf je pak vložen do tkinter okna přes FigureCanvasTkAgg místo samostatného plt.show():

```
def _kresli_graf(self, jmena):
    barvy   = ['#e74c3c', '#3498db', '#2ecc71', ...]  # 10 barev
    styly   = ['-', '--', '-.', ':']                  # 4 styly čar
    markery = ['o', 's', '^', 'D', ...]               # 8 markerů
```

```
    for jmeno in jmena:
        for pohlavi in df['Pohlaví'].unique():
            # groupby Rok -> průměr pořadí (pro případ více záznamů)
            sub = df[df['Pohlaví'] == pohlavi].groupby('Rok')['Pořadí'].mean()
            ax.plot(sub.index.astype(str), sub.values,
                    color=barvy[idx % len(barvy)],
                    linestyle=styly[idx % len(styly)],
                    marker=markery[idx % len(markery)])
            idx += 1

    ax.invert_yaxis()  # 1. místo nahoře, vyšší číslo = méně populární

    # Vložení do tkinter místo plt.show()
    canvas = FigureCanvasTkAgg(fig, master=win)
    toolbar = NavigationToolbar2Tk(canvas, win)  # lišta s nástoji
```

Zoom kolečkem myši je implementován jako callback funkce navázaná na událost MouseWheel. Mění limity osy Y o 15 % v závislosti na směru otočení kolečka:

```
def zoom_koleckem(event):
    faktor = 0.85 if event.delta > 0 else 1.15  # +kolecko = priblizit
    ax.set_ylim([y * faktor for y in ax.get_ylim()])
    canvas.draw()  # překreslit canvas


canvas.get_tk_widget().bind('<MouseWheel>', zoom_koleckem)
# Linux: Button-4 = kolečko nahoru, Button-5 = dolů
canvas.get_tk_widget().bind('<Button-4>', lambda e: zoom_koleckem(...))
```

11. Výpočet šance setkání – metoda odhadni_podil

Pravděpodobnost, že náhodná osoba daného pohlaví nese konkrétní jméno, je odhadnuta exponenciální funkcí kalibrovanou na česká data: 1. místo ~3 %, každé další místo exponenciálně klesá:

```
def odhadni_podil(self, poradi, max_poradi):
    # Exponenciální aproximace: top1 ~3%, top10 ~1%, top100 ~0.1%
    podil = 3.0 * math.exp(-0.045 * (poradi - 1))
    if poradi >= max_poradi:
        podil *= 0.5   # jména na posledním místě mají nižší podíl
    return max(podil, 0.01)  # minimum 0.01 %
```

Šance potkání v místnosti N lidí vychází z pravděpodobnosti, že ŽÁDNÝ z N lidí jméno nenese, a vezmeme doplněk:

```
sance_celkem = podil / 2        # 50/50 pohlaví v populaci
# P(alespon 1 z N) = 1 - P(zadny z N)
sance_20 = 1 - (1 - sance_celkem/100) ** 20
sance_50 = 1 - (1 - sance_celkem/100) ** 50
```

12. Dialog pro výběr roku – metoda vyber_rok_okno

Vytvoří modální okno s tlačítky pro každý dostupný rok. Technika s IntVar a wait_window() je standardní tkinter přístup pro blokující dialog – hlavní smyčka čeká dokud se okno nezavře, pak přečte výsledek z proměnné:

```
def vyber_rok_okno(self, titul):
```

```
    res = tk.IntVar(value=0)    # sem se uloží vybraný rok
    win = tk.Toplevel(self.root)

    for r in self.seznam_roku:
        # lambda k=r: zachytí aktuální hodnotu r (bez toho by všechna
        # tlačítka použila poslední hodnotu r po skončení smyčky)
        tk.Button(win, text=str(r),
                  command=lambda k=r: [res.set(k), win.destroy()]
        ).pack()

    self.root.wait_window(win)  # čekej dokud se okno nezavře
    return res.get() if res.get() != 0 else None
```

Závěr

Aplikace splňuje všechny stanovené cíle: načítá a správně parsuje oba formáty excelových souborů CSU, zobrazuje interaktivní grafy historického vývoje popularity jmen, umožňuje srovnávání jmen a výpočet pravděpodobnosti setkání.

Největší technická výzva bylo správné párování sloupců ve Formátu B a zpracování rozsahových pořadí (11–15 -> průměr 13). Obojí bylo vyřešeno pozičním párováním sloupců a průměrováním rozsahů namísto jejich expandování.

Možná budoucí rozšíření: porovnání s daty ze Slovenska, export výsledků do CSV, predikce trendu pomocí lineární regrese.

Zdroje a použitá literatura

- Cesky statisticky urad – data o jmenech: https://www.czso.cz/csu/czso/jak_se_dnes_jmenuji_deti

Kontakt: petr.saloun@upol.cz